Telco Customer Churn Prediction

Project Report

Python • pandas • numpy • scikit-learn • matplotlib • MLflow

Prepared on 09 April 2026

Executive Summary

The project built and compared several supervised learning models for Telco customer churn prediction using a local uv-based Python environment and MLflow tracking. After data cleaning, one-hot encoding, and light feature engineering, Logistic Regression proved to be a strong linear baseline, Random Forest improved precision/F1, and HistGradientBoosting achieved the strongest recall and ROC-AUC on the selected hold-out run. Cross-validation, however, showed that Logistic Regression and HistGradientBoosting generalize at a very similar level (mean CV F1 around 0.579), which means the final choice is driven more by operating point behavior than by a large generalization gap. The ensemble of Random Forest and HistGradientBoosting did not improve the main metrics.

1. Project Objective

The goal of this project was to predict whether a Telco customer would churn using a binary classification workflow implemented with Python, pandas, numpy, scikit-learn, matplotlib, and MLflow. The emphasis was on a clean Kaggle-style process: inspect the data, preprocess it, establish baselines, compare models fairly, and select a final model with clear reasoning. We understood that recall for churned customers should be given highest priority, because missing churners is costly, and that we shouldn't rely on accuracy alone, because the dataset is imbalanced.

2. Dataset Overview

The working dataset contained 7,032 cleaned customer records and 23 raw columns after preprocessing removed incomplete rows. The target variable was Churn, encoded as a binary label.

Item	Value	Notes
Raw rows	7,032	After cleaning
Raw columns	23	Original Telco schema
Encoded feature columns	36	After get_dummies and feature engineering
Target distribution	5,163 non-churn / 1,869 churn	Moderate class imbalance

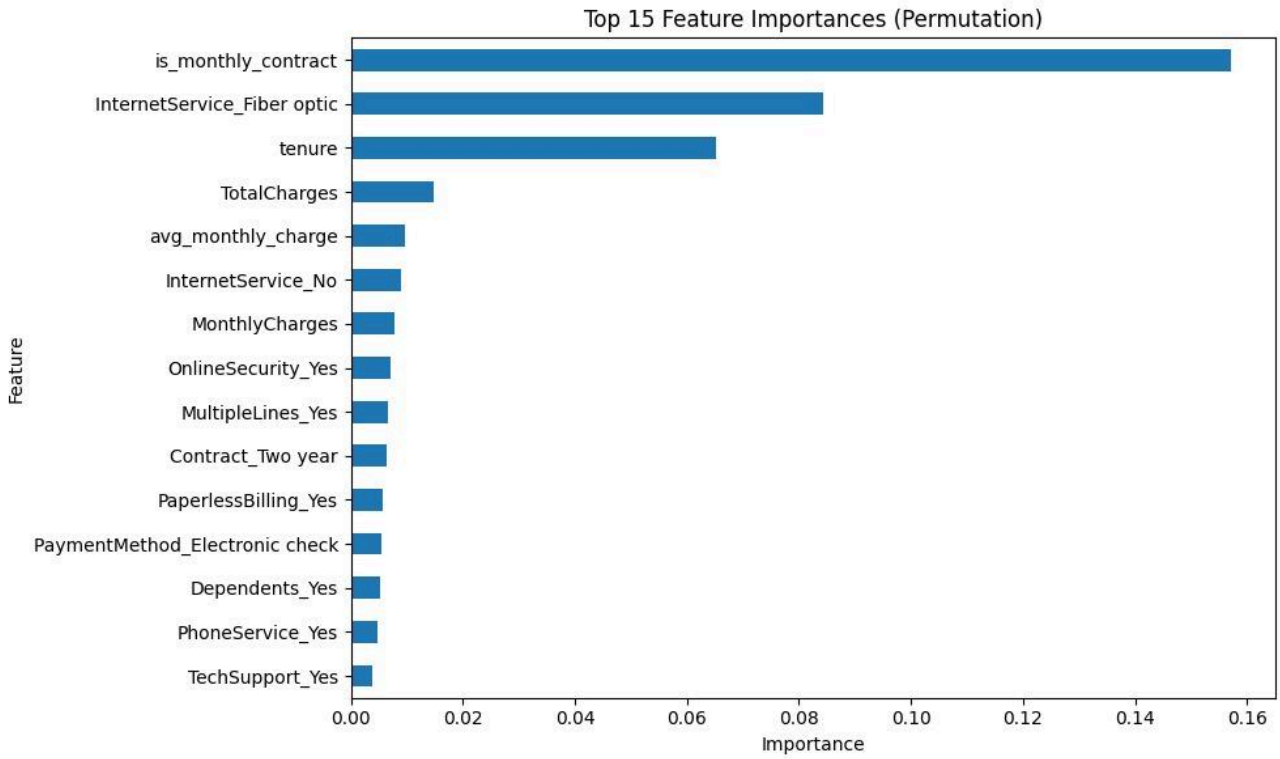
3. Preprocessing and Feature Engineering

The preprocessing pipeline was implemented in preprocess.py and invoked from train.py. The code path was kept simple and reproducible so that every model saw the same feature matrix.

- Cleaned the raw dataset and removed rows with unusable values.

- Encoded categorical variables with pandas.get_dummies.
- Added avg_monthly_charge as a derived ratio feature.
- Added tenure_group by binning tenure into interpretable ranges.
- Added is_monthly_contract as a binary indicator for contract type.
- Tried dropping the StreamingTV and StreamingMovies during feature selection but found it only worsened the score metrics (decision to drop these columns was based on feature importance calculated using model.feature_importances_ function (StreamingTV and StreamingMovies were the bottom-most features with least importance))

An ablation test showed that the engineered features were helpful for model consistency but did not create a large jump in performance for the linear baseline. In particular, Logistic Regression performed almost identically with and without the engineered features, which suggests the encoded categorical structure already captured most of the available signal.

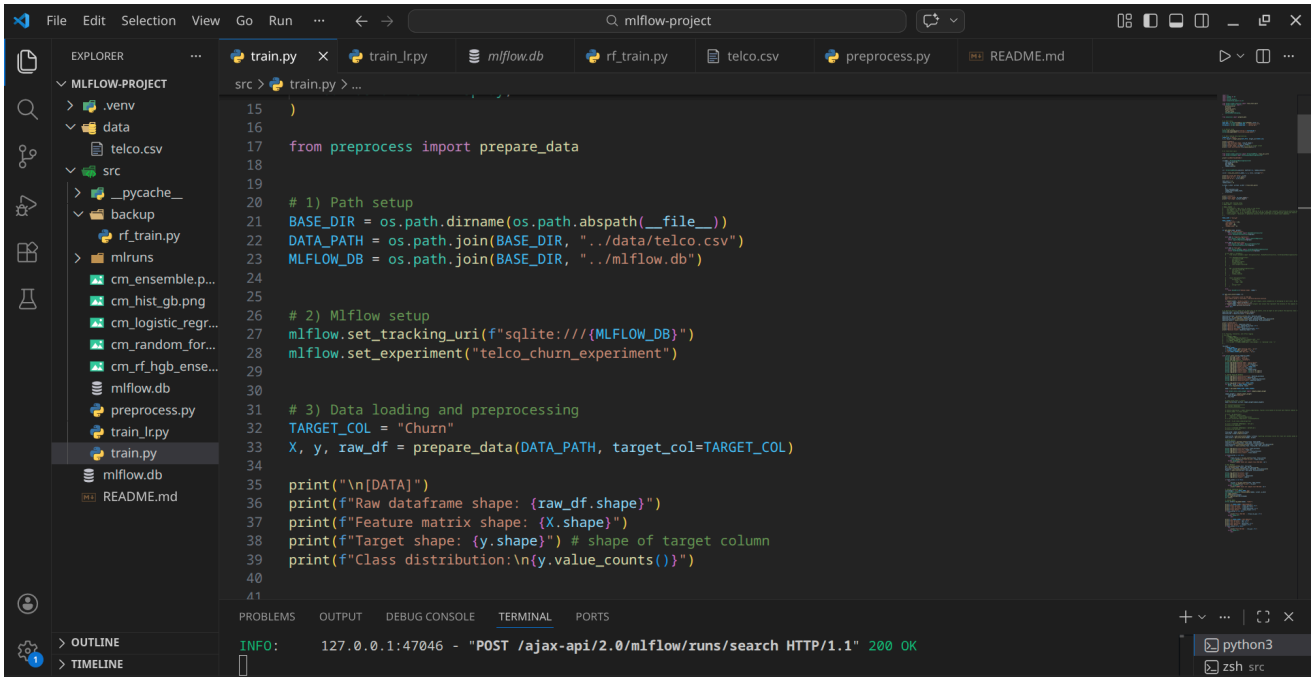


4. Experimental Setup

- Train/test split: 80/20 with stratification and random_state = 42.
- Primary metrics: accuracy (can be misleading as dataset is imbalanced), precision, recall, F1, ROC-AUC.
- All runs were tracked in MLflow using a local SQLite backend (mlflow.db).
- Artifacts included confusion matrices and saved models.
- Imbalance handling used class_weight='balanced' or sample_weight where applicable.

A majority-class baseline was also computed. It achieved roughly 73.4% accuracy but zero precision, zero recall, and zero F1 on the positive churn class, so any useful model had to substantially outperform that trivial strategy.

VSCode Setup:



5. Models Evaluated

Model	Key Settings	Role in the Workflow
Logistic Regression	max_iter=1000, solver=liblinear, class_weight=balanced	Linear baseline and interpretability reference
Random Forest	n_estimators=100, max_depth=14, min_samples_leaf=5, class_weight=balanced	Strong tree-based benchmark
HistGradientBoosting	learning_rate=0.03, max_depth=6, max_iter=100	Final candidate and selected model
Soft Voting Ensemble	RF + HGB	Experimentally tested; rejected

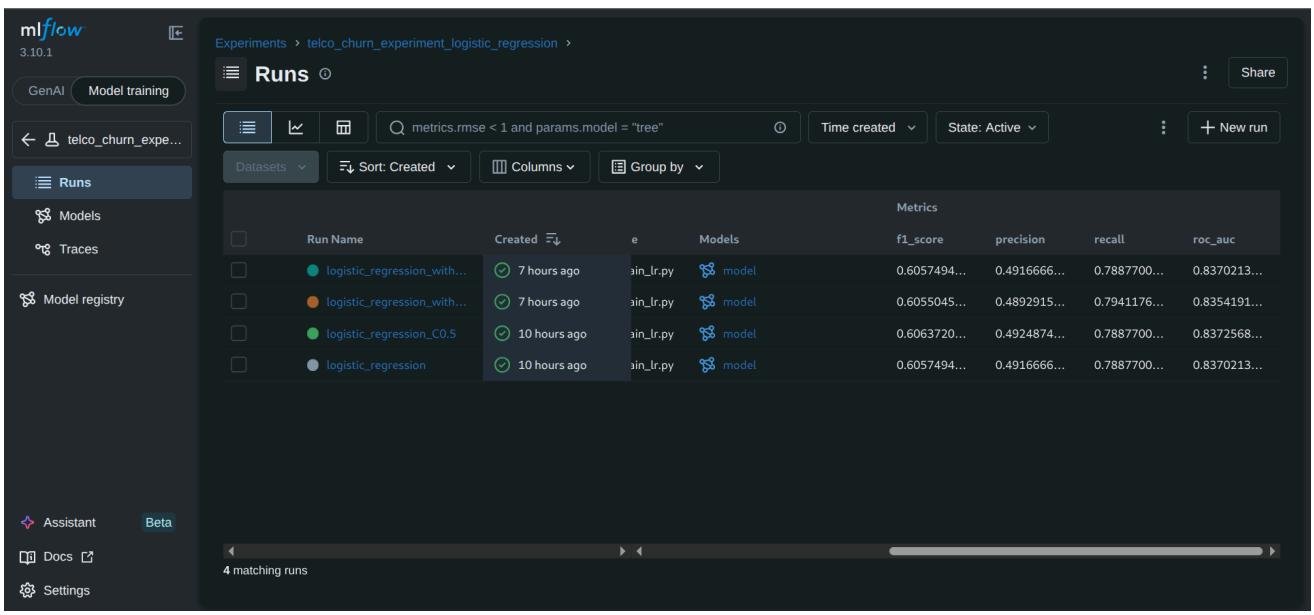
6. Results and Comparison

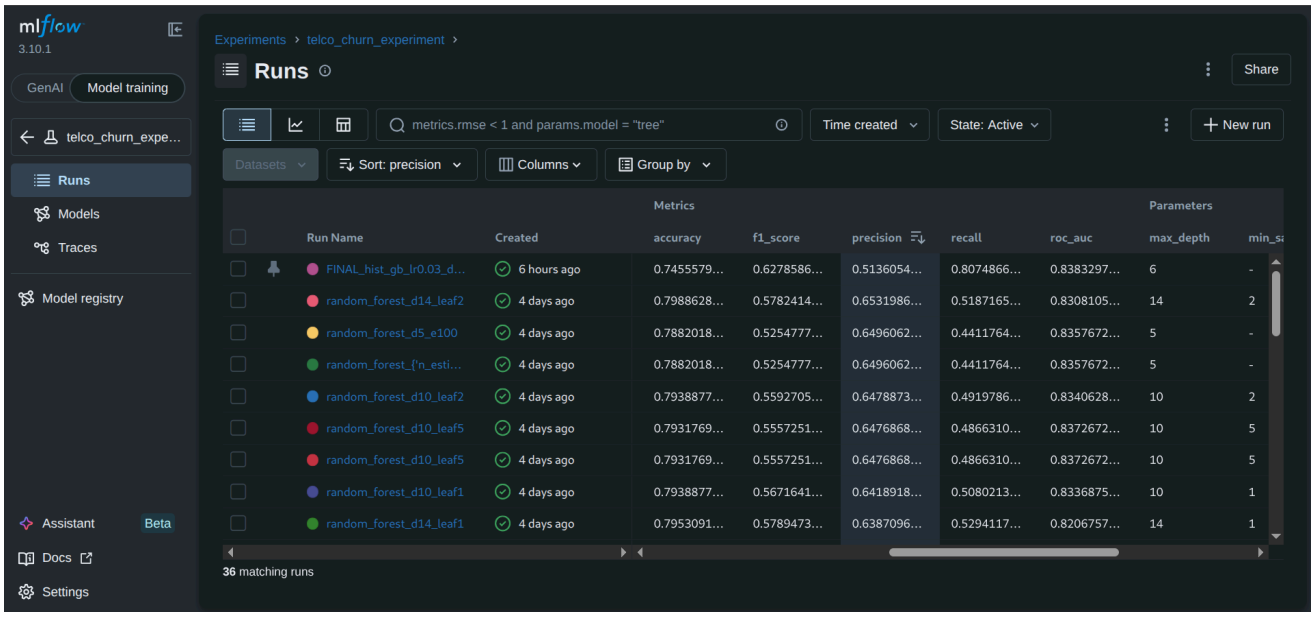
The following table summarizes the main runs discussed during experimentation. The numbers below reflect the selected operating points that were used to compare models and assess the precision-recall tradeoff.

Model / run	F1	Precision	Recall	ROC-AUC
Logistic Regression	0.6057	0.4917	0.7888	0.8370
Random Forest	0.6281	0.5453	0.7406	0.8338
HistGB (default threshold)	0.6279	0.5136	0.8075	0.8383
HistGB (threshold tuned)	0.6295	0.5752	0.6952	0.8383
Soft Voting Ensemble	0.6138	0.5975	0.6310	0.8371

Interpretation: Logistic Regression was already strong, which indicates that the feature space is close to linearly separable after one-hot encoding. Random Forest improved F1 and precision relative to Logistic Regression, while HistGradientBoosting offered the best recall and the strongest ROC-AUC on the chosen test split. The ensemble reduced recall materially and therefore reduced F1, so it was discarded.

Running in Mlflow

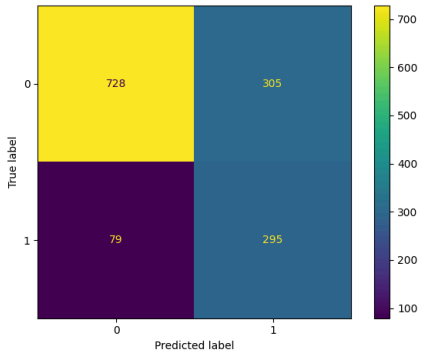




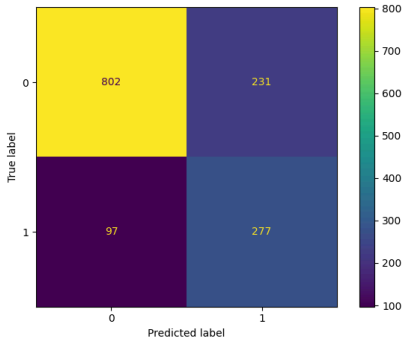
		Metrics					Parameters	
Run Name	Created	accuracy	f1_score	precision	recall	roc_auc	max_depth	min_sa
FINAL_hist_gb_lr0.03_d...	6 hours ago	0.7455579...	0.6278586...	0.5136054...	0.8074866...	0.8383297...	6	-
random_forest_d14_leaf2	4 days ago	0.7988628...	0.5782414...	0.6531986...	0.5187165...	0.8308105...	14	2
random_forest_d5_e100	4 days ago	0.7882018...	0.5254777...	0.6496062...	0.4411764...	0.8357672...	5	-
random_forest_{'n_esti...	4 days ago	0.7882018...	0.5254777...	0.6496062...	0.4411764...	0.8357672...	5	-
random_forest_d10_leaf2	4 days ago	0.7938877...	0.5592705...	0.6478873...	0.4919786...	0.8340628...	10	2
random_forest_d10_leaf5	4 days ago	0.7931769...	0.5557251...	0.6476868...	0.4866310...	0.8372672...	10	5
random_forest_d10_leaf5	4 days ago	0.7931769...	0.5557251...	0.6476868...	0.4866310...	0.8372672...	10	5
random_forest_d10_leaf1	4 days ago	0.7938877...	0.5671641...	0.6418918...	0.5080213...	0.8336875...	10	1
random_forest_d14_leaf1	4 days ago	0.7953091...	0.5789473...	0.6387096...	0.5294117...	0.8206757...	14	1

Confusion matrices for each model

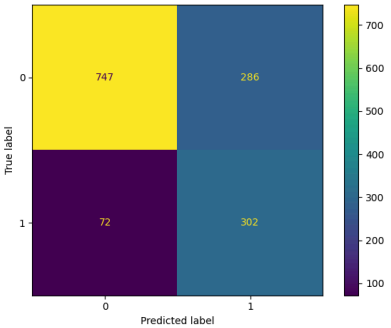
Logistic Regression



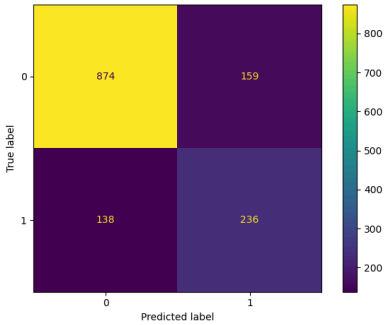
Random forest



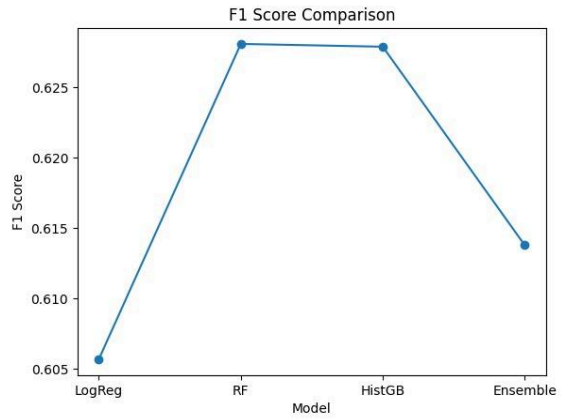
HistGB

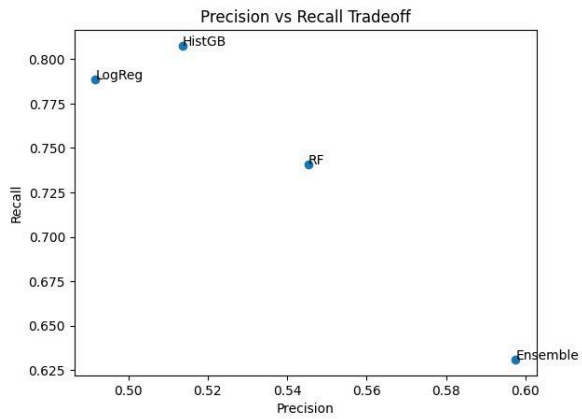


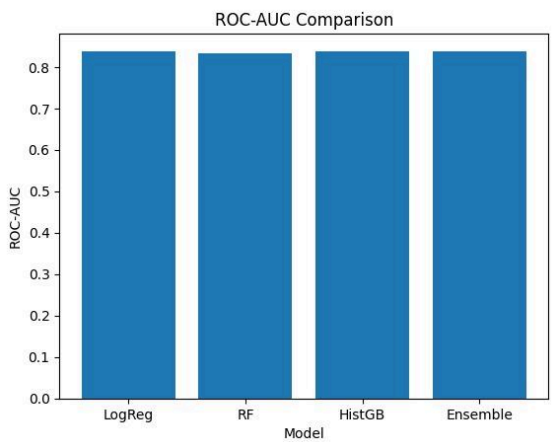
Ensemble (Random forest and HistGB)

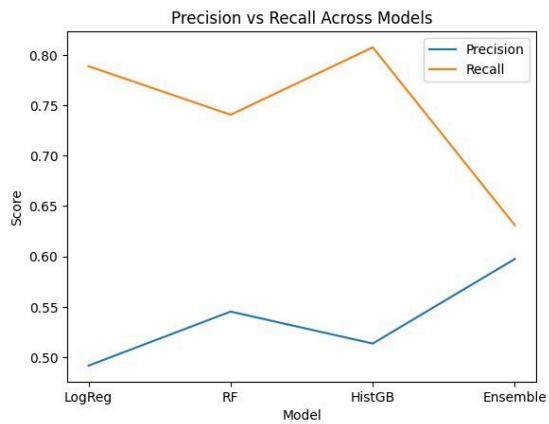


Score metrics Comparison









7. Cross-Validation Findings

To check whether the hold-out test split was overly optimistic, 5-fold stratified cross-validation was run for the main contenders. This revealed that the single test split slightly overstated performance.

Model	Mean CV F1	Std. Dev.
Logistic Regression	0.5787	0.0167
HistGradientBoosting	0.5787	0.0167

The cross-validation result is important: Logistic Regression and HistGradientBoosting generalize at essentially the same level under repeated stratified splits.HistGradientBoosting was selected due to its ability to better capture non-linear feature interactions and its flexibility in adjusting the precision–recall tradeoff through threshold tuning. In practice, it demonstrated stronger recall at certain operating points, which is desirable for churn prediction where identifying potential churners is critical.

8. Threshold Tuning Analysis

Threshold tuning was tested as a post-processing step for HistGradientBoosting. The default 0.5 threshold produced higher recall, while the tuned threshold slightly increased precision and F1 but reduced recall more than desired.

Setting	F1	Precision	Recall
Default threshold	0.6279	0.5136	0.8075
Tuned threshold	0.6295	0.5752	0.6952

For a churn problem, recall is usually more important than a small gain in precision. Because the tuned threshold reduced recall sharply, the default threshold was retained as the preferred operating point.

9. Final Model Selection

- Selected model: HistGradientBoostingClassifier.
- Selected operating point: default threshold of 0.5.
- Reason: best balance for churn intervention, with the strongest recall among the main finalists and top ROC-AUC.
- Logistic Regression remains a strong benchmark and should be kept in the report as a practical baseline.

The ensemble model was rejected because it suppressed recall and lowered F1. Decision Tree was not pursued further because it would be a strictly weaker version of the tree-based models already tested.

10. Conclusion

This project produced a complete churn modeling workflow with reproducible preprocessing, MLflow tracking, and cross-validation. The feature-engineered representation contains a strong churn signal: Logistic Regression is competitive, Random Forest improves F1, and HistGradientBoosting provides the strongest recall-oriented operating point.

Report by:

Siddharth S - 24011103059

K Tejas Reddy - 24011103025